

COEN 317 Lab Quiz Offline Preparation Guide

Built from your old prep PDF, the Lab 3 manual, the old quiz example, and the fixes we worked out together.

Most likely quiz setup: 1 AXI GPIO + 1 AXI Timer + 2 LEDs on E15/D15. The output should cycle through 00, 01, 10, 11.

1. What the quiz is most likely asking

Based on everything you collected, the quiz is most likely a Lab 3 Part 1 style task:

- Create a small hardware system in XPS with one AXI GPIO and one AXI Timer.
- Map two GPIO output bits to two board LEDs.
- Write a short SDK C++ program that uses the timer count to drive a repeating 2-bit LED pattern.
- The safest expected visible pattern is 00 -> 01 -> 10 -> 11 -> repeat.

2. Keep these facts in your head during the quiz

Item	What to use
Best lab to follow	Lab 3 Part 1 for hardware flow, UCF naming, AXI GPIO, and AXI Timer.
Two instances	Most likely means one AXI GPIO instance and one AXI Timer instance.
Safest GPIO width	2 bits only, unless the quiz explicitly asks for more.
Best external output port name	output
Best timer instance name	axi_timer_0
Best GPIO instance name	axi_gpio_for_output
Two LED pins	E15 and D15
SDK terminal	/dev/ttyUSB0, 115200, 8 data bits, 1 stop bit, no parity, no flow control.
Bitstream backup fix	-g UnconstrainedPins:Allow

3. Exact hardware flow in PlanAhead and XPS

1. Create a new RTL project.
2. Target language: VHDL.
3. Board: ZYNQ-7 ZC702.
4. Add your UCF file before finishing project creation.
5. Add or Create Embedded Sources.
6. Create sub-design named system.
7. Let XPS open through the BSB wizard.
8. Remove the default GPIO_SW and LEDs_4Bits peripherals.
9. Add AXI GPIO. Rename instance to axi_gpio_for_output.
10. Set GPIO width to 2 bits.
11. Add AXI Timer/Counter. Rename instance to axi_timer_0.

12. If you want old-quiz timing behavior, set timer width to 26 bits.
13. In the Ports tab, under axi_gpio_for_output -> (IO_IF) gpio_0, set GPIO_IO to No Connection.
14. Then set GPIO_IO_O to Make External.
15. Rename the external port to output exactly.
16. Generate addresses so both axi_gpio_for_output and axi_timer_0 get valid base/high addresses.
17. Check Bus Interfaces. Both GPIO and timer must be connected on AXI and must not say No Connection.
18. Save all, exit XPS, create top HDL, run synthesis, run implementation, generate bitstream, program FPGA, export hardware to SDK.

4. The UCF file you should keep ready

Use this outputs-only UCF unless the quiz sheet explicitly gives you input pins too.

```
NET output<0> LOC = E15;
NET output<1> LOC = D15;

NET output[*] IOSTANDARD = LVCMOS25;
```

Important rule:

The external top-level port in system_stub.vhd and in XPS must be named output exactly. If the UCF says output but the external port has another name, the LEDs will not behave correctly.

5. What system_stub.vhd must look like

You do not usually edit this file by hand. But after Create Top HDL, check that the top-level entity contains this exact kind of port:

```
output : out std_logic_vector(1 downto 0)
```

If you see that line, your UCF name and top-level port width are consistent.

6. Best main.cc to keep ready

This is the corrected version based on your friend's note that the board should show the four 2-bit patterns 00, 01, 10, and 11.

```
#include "xparameters.h"
#include "xil_types.h"
#include "xgpio.h"
#include "xstatus.h"
#include <iostream>

using namespace std;

#define PATTERN_00 0x00
#define PATTERN_01 0x01
#define PATTERN_10 0x02
#define PATTERN_11 0x03

int main()
{
    static XGpio GpioOut;
    volatile u32* Timer_Ptr = (u32*) XPAR_TMRCTR_0_BASEADDR;
```

```

int status;

cout << "COEN 317 Lab Quiz Application Started" << endl;

status = XGpio_Initialize(&GpioOut, XPAR_AXI_GPIO_FOR_OUTPUT_DEVICE_ID);
if (status != XST_SUCCESS)
{
    cout << "GPIO output initialization FAILED" << endl;
    return 1;
}

XGpio_SetDataDirection(&GpioOut, 1, 0x0);

*Timer_Ptr = 0x001;
*(Timer_Ptr + 1) = 0;
*Timer_Ptr = 0x021;
*Timer_Ptr = 0x001;
*Timer_Ptr = 0x081;

cout << "Timer started" << endl;

while (1)
{
    u32 raw_count = *(Timer_Ptr + 2);

    // Good visible speed at 50 MHz
    u32 state = (raw_count >> 25) & 0x3;

    u32 led_value;

    switch (state)
    {
        case 0: led_value = PATTERN_00; break;
        case 1: led_value = PATTERN_01; break;
        case 2: led_value = PATTERN_10; break;
        case 3: led_value = PATTERN_11; break;
        default: led_value = PATTERN_00; break;
    }

    XGpio_DiscreteWrite(&GpioOut, 1, led_value);
}

return 0;
}

```

Why this version is better:

- It shows all four 2-bit states, not just one LED, off, the other LED, off.
- It does not use unstable threshold toggling that can flip too fast near wraparound.
- It gives a visible board pattern at about two-thirds of a second per state when the timer clock is 50 MHz.

7. If the pattern is too fast or too slow

Only change one line in main.cc:

```
u32 state = (raw_count >> 25) & 0x3;
```

Try these alternatives:

Need	Line to use
Slower	u32 state = (raw_count >> 26) & 0x3;

Faster	u32 state = (raw_count >> 24) & 0x3;
--------	--------------------------------------

8. Quick board test sequence

If you are unsure whether the board output is working, test in this order.

Static test 1

```
XGpio_DiscreteWrite(&GpioOut, 1, 0x01);  
cout << "LED0 should be ON" << endl;  
while (1);
```

Expected result: Only one LED should stay on.

Static test 2

```
XGpio_DiscreteWrite(&GpioOut, 1, 0x02);  
cout << "LED1 should be ON" << endl;  
while (1);
```

Expected result: The other LED should stay on.

Static test 3

```
XGpio_DiscreteWrite(&GpioOut, 1, 0x03);  
cout << "Both LEDs should be ON" << endl;  
while (1);
```

Expected result: Both LEDs should stay on.

If all three static tests work, your UCF, GPIO connection, external port naming, and board path are correct. Then return to the timer-based quiz code.

9. What to look for on the board

- Do not judge success just because some random board LEDs are lit. Some LEDs are only board status or cable status LEDs.
- Judge success only by the two LEDs connected to your UCF pins E15 and D15.
- Correct quiz behavior means the two quiz LEDs should visibly cycle through 00, 01, 10, 11.

10. SDK checklist

- Create an Empty Application project.
- Paste main.cc into src.
- Open xparameters.h and verify the macro names used in main.cc.
- The two most important macros are usually XPAR_AXI_GPIO_FOR_OUTPUT_DEVICE_ID and XPAR_TMRCTR_0_BASEADDR.
- If XPS generated slightly different names, change the macro names in main.cc to match xparameters.h exactly.
- Build the project.

- Open the serial terminal on /dev/ttyUSB0 at 115200 baud.
- Run the application on hardware.

11. If SDK or Xilinx gives errors

Problem	Best fix
Bitstream generation complains about unconstrained pins	Open Bitstream Settings, then in More Options type: -g UnconstrainedPins:Allow
Program runs in terminal but LEDs do nothing	Check that GPIO_IO_O is the external signal, not GPIO_IO. Check that external port name is output. Check that the UCF also uses output.
Build fails because a macro is not found	Open xparameters.h and copy the actual generated macro names into main.cc.
Weird XMD or debugger target errors	Terminate old run sessions, power cycle the board, program FPGA again, restart SDK, clean project, rebuild, and use Run As instead of Debug first.
Board looks alive but you are not sure the app is working	Use the static GPIO tests 0x01, 0x02, and 0x03 before blaming the timer logic.

12. Very short quiz-day checklist

- New project -> VHDL -> ZC702 -> add UCF.
- Create sub-design system and open XPS.
- Remove default GPIO_SW and LEDs_4Bits.
- Add axi_gpio_for_output width 2.
- Add axi_timer_0.
- Set GPIO_IO to No Connection and GPIO_IO_O to Make External.
- Rename external port to output.
- Generate addresses.
- Check Bus Interfaces.
- Create Top HDL, run synthesis, implementation, bitstream, program FPGA.
- Export to SDK, create Empty Application, paste main.cc, build, run.
- Look for the pattern 00 -> 01 -> 10 -> 11 on the two quiz LEDs.

13. Final advice

Start simple. Get one working output path first. Do not add inputs, capture mode, PWM, or interrupts unless the quiz paper explicitly asks for them. The fastest path to success is one AXI GPIO, one AXI Timer, two output bits, a clean UCF, and a visible four-state board pattern.